**INF201 - Deliverable 2: Kachina Aschenberg Øwre & Seline Heggelund Knutsen**

Link to our github repository: https://github.com/kacoew/INF201-CAs

# Introduction

This notebook builds a data pipeline for the Campus Coffee Shop chain. It finds all register log files, extracts valid sales lines, cleans price errors and creates summary reports (Tasks 1–4 build the functions, Task 5 combines them).

**Before running:** generate the input data with `python generate_data_for_d2.py`. The `data/` folder is not in the GitHub repository. All output files are saved in the `output/` folder.

In [1]:
```python
import json # for reading JSON files
import re # for regular expressions
from pathlib import Path # for file paths
import numpy as np # for numerical operations
import pandas as pd # for data handling and analysis

DATA_DIR = Path("data")            # relative path -> works on both laptops
LOG_ROOT = DATA_DIR / "2026"
CONFIG_FILE = DATA_DIR / "store_config.json"
OUTPUT_DIR = Path("output")
EPSILON = 1e-6                     # Task 3.2: avoids division by zero
```

# Task 1.1 – Robust file reader

- Use `pathlib.Path`.
- Handle both `UTF-8` and `ISO-8859-1` without hard-coding which file has which.
- Return `(file_text_content, encoding_used)`.

**Our solution:** We do not know which encoding a file has, so the function tries to read it as UTF-8. If that fails with a `UnicodeDecodeError`, it reads the file again as ISO-8859-1 (Latin-1). The order matters, Latin-1 accepts every possible byte and never fails, so if we tried it first, UTF-8 files with special characters would be read incorrectly without any error. We only catch `UnicodeDecodeError`, so other problems (e.g. a wrong file path) still raise an error.

In [2]:
```python
# Task 1.1: read_text_file

def read_text_file(file_path: str | Path) -> tuple[str, str]:
    """Read a text file whose encoding is either UTF-8 or ISO-8859-1.

    Returns a tuple (file_text_content, encoding_used).
    """
    path = Path(file_path)
    try:
        return path.read_text(encoding="utf-8"), "utf-8"
    except UnicodeDecodeError:  # If UTF-8 raises UnicodeDecodeError, read as ISO-8859-1 instead
        # Latin-1 maps every byte to a character, so this fallback cannot fail.
        return path.read_text(encoding="iso-8859-1"), "iso-8859-1"
```

In [3]:
```python
# Test both branches (the real log files are plain ASCII, so they only hit UTF-8).
test_dir = OUTPUT_DIR / "encoding_test"
test_dir.mkdir(parents=True, exist_ok=True)

(test_dir / "latin1.txt").write_text("Café Crème", encoding="iso-8859-1")
(test_dir / "utf8.txt").write_text("Café Crème", encoding="utf-8")

print(read_text_file(test_dir / "latin1.txt"))
print(read_text_file(test_dir / "utf8.txt"))
```

```
('Café Crème', 'iso-8859-1')
('Café Crème', 'utf-8')
```

**What the test shows:** The generated log files only contain ASCII characters, so they are always read with UTF-8 and never test the fallback. We therefore wrote "Café Crème" to two test files with different encodings. Both are read correctly, and the function reports the correct encoding for each, which shows that both branches work.

# Task 1.2 – Log parser with regular expressions

Valid line: `[2026-10-15 08:30:00] [POS_01] item=Latte price=4.50 EUR (SUCCESS)`

Noise to ignore, e.g. `--- PRINTER PAPER LOW ---`, `=== NETWORK SYNC OK ===`, `[DEBUG] BARCODE SCANNER REBOOTED`.

Capture 6 fields: `timestamp`, `register_id`, `item`, `price` (float), `currency`, `status`. Return a list of dicts.

**Our solution:** We use one regular expression with a named group for each of the six fields:

Example: `[2026-10-15 08:30:00] [POS_01] item=Latte price=4.50 EUR (SUCCESS)`

| Group | Value in the example | Pattern |
|---|---|---|
| `timestamp` | `2026-10-15 08:30:00` | `\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}` |
| `register_id` | `POS_01` | `\w+` |
| `item` | `Latte` | `\w+` |
| `price` | `4.50` | `\d+\.\d+` |
| `currency` | `EUR` | `[A-Z]{3}` |
| `status` | `SUCCESS` | SUCCESS, REFUND or CANCELLED |

In [4]:
```python
# SALES_LINE is a regex pattern that matches a sales line in the log file.
# List of strings:
# - timestamp: format YYYY-MM-DD HH:MM:SS
# - register_id: word characters (letters, digits, and underscores)
# - item: word characters (letters, digits, and underscores)
# - price: digits followed by a decimal point and more digits
# - currency: three uppercase letters
# - status: either 'SUCCESS', 'REFUND', or 'CANCELLED'

SALES_LINE = re.compile(
    r"^\[(?P<timestamp>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\] "
    r"\[(?P<register_id>\w+)\] "
    r"item=(?P<item>\w+) "
    r"price=(?P<price>\d+\.\d+) "
    r"(?P<currency>[A-Z]{3}) "
    r"\((?P<status>SUCCESS|REFUND|CANCELLED)\)$",
    re.MULTILINE, # MULTILINE flag allows the regex to match each line in the text separately
)

def parse_log(text: str) -> list[dict]:
    """Return one dict per valid sales line in ``text``; noise lines are skipped.
    Each dict has keys: timestamp, register_id, item, price, currency, status.
    """
    records = []
    for match in SALES_LINE.finditer(text):
        record = match.groupdict()
        record["price"] = float(record["price"])
        records.append(record)
    return records
```

In [5]:
```python
sample = """[2026-10-15 08:30:00] [POS_01] item=Latte price=4.50 EUR (SUCCESS)
--- PRINTER PAPER LOW ---
[DEBUG] BARCODE SCANNER REBOOTED
=== NETWORK SYNC OK ==="""

result = parse_log(sample)
print(len(result))   # expected: 1
print(result)
```

```
1
[{'timestamp': '2026-10-15 08:30:00', 'register_id': 'POS_01', 'item': 'Latte', 'price': 4.5, 'currenc
y': 'EUR', 'status': 'SUCCESS'}]
```

**What the test shows:** Of the four test lines, only the valid sales line is returned, and its price is the float `4.5`. The line `[DEBUG] BARCODE SCANNER REBOOTED` is an important test case, because it starts with `[` just like a sales line. It is still skipped, because the full pattern does not match.

## Task 2.1 – Recursive utility functions

Use `Path.iterdir()` and explicit recursion. **Not allowed:** `glob`, `rglob`, `os.walk`.

**Our solution:** The function calls itself on every subfolder; we sort the result because `iterdir()` gives no fixed order.

In [6]:
```python
def find_log_files(directory: str | Path, extension: str = ".log") -> list[Path]:
    """Find all log files in nested directories.

    Recursively collect every file with ``extension`` below ``directory``.

    Return a sorted list of Path objects.
    """
    directory = Path(directory)
    found: list[Path] = []
    for child in directory.iterdir(): # iterate over all files and directories in the given directory
        if child.is_dir(): # if the child is a directory, recursively call find_log_files on it
            found.extend(find_log_files(child, extension))
        elif child.is_file() and child.suffix == extension: # if the child is a file and has the given
            found.append(child)
    return sorted(found)   # fixed order -> reproducible output
```

In [7]:
```python
# Find all log files in the LOG_ROOT directory and its subdirectories.
log_files = find_log_files(LOG_ROOT)
print(len(log_files))
print(log_files[0])
```

```
9
data/2026/library_cafe/POS_01/register_sales.log
```

**What the test shows:** The function finds 9 log files: 3 stores × 3 registers, with one `register_sales.log` in each register folder. This matches the folder structure created by `generate_data_for_d2.py`.

## Task 2.2 – Active register IDs from nested JSON

Traverse any combination of nested dicts and lists. Return `set[str]` of every register `id` where `"active": true`.

**Our solution:** Text, numbers and booleans are the base case; we return a `set`, so an ID that appears several times is only stored once.

In [8]:
```python
def find_active_registers(node) -> set[str]:
    """Recursively collect the IDs of all active registers in a nested config.

    ``node`` can be any mix of dicts and lists (e.g. the loaded
    store_config.json). A dict counts as an active register if it has an
    "id" key and "active" is True.

    Returns a set of register ID strings, e.g. {"POS_01", "POS_02"}.
    """
    active: set[str] = set()
    if isinstance(node, dict):
        if node.get("active") is True and "id" in node:
            active.add(node["id"])
        for value in node.values():
            active |= find_active_registers(value)
    elif isinstance(node, list):
        for element in node:
            active |= find_active_registers(element)
    return active
```

In [9]:
```python
# Test the function with the store_config.json file.
config = json.loads(CONFIG_FILE.read_text(encoding="utf-8"))
print(find_active_registers(config))   # expected {'POS_01', 'POS_02', 'POS_03'}
```

```
{'POS_02', 'POS_03', 'POS_01'}
```

**What the test shows:** The active registers are POS_01, POS_02 and POS_03. POS_04 is not included, because it has `"active": false`. All register IDs that appear in the log files (POS_01–POS_03) are active, so filtering on active registers would not remove any transactions. Note that the config only lists two stores (no Student Union), and the same ID (e.g. POS_01) is used in several stores, so a register ID alone does not identify one specific register.

## Task 3.1 – Remove price outliers

Convert to a 1D float array, flag values with $|x - \mu| > 3\sigma$, replace them with the median of the valid prices.

**Our solution:** The prices are first converted to a 1D float NumPy array. We use `np.array`, which makes a copy, so the original data is not changed. We find all outliers at once with a boolean mask and replace them with the median of the remaining (valid) prices.

In [10]:
```python
def remove_price_outliers(prices) -> np.ndarray:
    """Replace prices with |x - mean| > 3 * std by the median of the other prices.

    Returns a new 1D float array; the input is not changed.
    """
    values = np.array(prices, dtype=float).ravel()  # np.array makes a COPY - pandas 3 may give a read-
    mu, sigma = values.mean(), values.std()
    is_outlier = np.abs(values - mu) > 3 * sigma
    values[is_outlier] = np.median(values[~is_outlier])
    return values
```

In [11]:
```python
# Test with MANY normal values + one glitch (with only 2-3 values,
# sigma is so large that 999 is NOT > 3 sigma):

test = [4.0, 4.5, 5.0, 3.5, 4.2, 4.8, 3.9, 4.1, 4.6, 4.4, 999.0]
print(remove_price_outliers(test))
```

```
[4.  4.5 5.  3.5 4.2 4.8 3.9 4.1 4.6 4.4 4.3]
```

**What the test shows:** In the test list, 999.0 is replaced by 4.3, the median of the ten normal prices. On our real data (720 prices), the mean is 45.86 and the standard deviation is 198.75, both strongly inflated by the glitches. The limit is therefore about 642, but 999.00 is still far above it, so all 30 glitch values are detected. Afterwards the highest price is 8.00.

**Limitation:** The 3σ rule only works because 999 is so extreme. With very few values (e.g. `[4, 5, 999]`) the standard deviation becomes so large that 999 is not detected, and a smaller glitch (e.g. 15.00) would not be caught. The median (3.97) is also calculated across all items and currencies, so a glitched Sandwich price gets roughly the price of an Espresso.

## Task 3.2 – compute_refund_ratio

$$\text{Refund Ratio} = \frac{\text{Refund Count}}{\text{Sales Count} + 10^{-6}}$$

**Our solution:** The function converts both inputs to float arrays and divides them in one NumPy operation, which works element by element, so no loop is needed. A small number $\epsilon = 10^{-6}$ is added to the sales count, so a count of 0 does not cause a division by zero.

In [12]:
```python
def compute_refund_ratio(sales_count: np.ndarray, refund_count: np.ndarray) -> np.ndarray:
    """Return refund_count / (sales_count + EPSILON), element by element.

    EPSILON = 1e-6 avoids division by zero.
    """
    sales = np.asarray(sales_count, dtype=float)
    refunds = np.asarray(refund_count, dtype=float)
    return refunds / (sales + EPSILON)     # no loop needed
```

In [13]:
```python
print(compute_refund_ratio(np.array([10, 0]), np.array([2, 0])))
```

```
[0.19999998 0.        ]
```

**What the test shows:** 2 refunds out of 10 sales gives 0.19999998 instead of exactly 0.2, because of ε. The difference is too small to matter. When both counts are 0, the result is 0 instead of an error or `NaN`, which shows that ε prevents division by zero.

## Task 4.1 – Build the DataFrame

List of dicts → DataFrame, `timestamp` as `datetime` index, sorted chronologically.

**Our solution:** The list of dictionaries is turned into a DataFrame with `pd.DataFrame`. The `timestamp` column is converted with `pd.to_datetime`, so pandas treats it as time and not as text, which is needed for the resampling in Task 4.2. It is then set as the index and sorted, because the log files are read one after another, so transactions from different registers are not in time order.

In [14]:
```python
def create_dataframe(records: list[dict]) -> pd.DataFrame:
    """Convert transaction records into a DataFrame

    Convert timestamps to datetime, set them as the index,
    and sort transactions chronologically
    """
```

```
    # Convert the list of dictionaries into a DataFrame
    df = pd.DataFrame(records)

    # Convert timestamps to datetime objects
    df["timestamp"] = pd.to_datetime(df["timestamp"])

    # Set timestamp as the index and sort chronologically
    df = df.set_index("timestamp")
    df = df.sort_index()

    return df
```

In [15]:
```
# Test on one real log file (80 transactions) instead of the one-line sample
test_text, _ = read_text_file(log_files[0])
df = create_dataframe(parse_log(test_text))
df["price"] = remove_price_outliers(df["price"])   # remove the 999.00 glitches (Task 3.1)

print(df)
print("Index type:", type(df.index).__name__, "| sorted:", df.index.is_monotonic_increasing)
```

```
                    register_id        item  price currency     status
timestamp
2026-10-15 07:01:21      POS_01  Cappuccino   3.83      USD  CANCELLED
2026-10-15 07:04:07      POS_01       Latte   4.47      EUR    SUCCESS
2026-10-15 07:04:36      POS_01      Muffin   3.21      NOK    SUCCESS
2026-10-15 07:06:18      POS_01       Latte   4.99      EUR    SUCCESS
2026-10-15 07:08:44      POS_01      Muffin   3.17      USD  CANCELLED
...                         ...         ...    ...      ...        ...
2026-10-15 09:01:51      POS_01      Muffin   3.88      NOK  CANCELLED
2026-10-15 09:03:14      POS_01      Muffin   3.23      NOK    SUCCESS
2026-10-15 09:05:51      POS_01      Muffin   3.96      USD    SUCCESS
2026-10-15 09:06:05      POS_01   Croissant   4.16      NOK     REFUND
2026-10-15 09:06:34      POS_01    Espresso   2.79      NOK    SUCCESS

[80 rows x 5 columns]
Index type: DatetimeIndex | sorted: True
```

**What the test shows:** We test on one real log file ( `library_cafe/POS_01` ) instead of the one-line sample, so the tables in Task 4 have more than one row. The 999.00 glitches are removed first with the function from Task 3.1, so they do not dominate the tables. The result has 80 rows and 5 columns, and the index is a `DatetimeIndex` running from 07:01 to 09:06, which shows that the conversion and sorting work.

## Task 4.2 – Hourly revenue per item

Group by item, resample in 1-hour windows, sum the price.

**Our solution:** We group the transactions by item and use `resample("1h")` on the time index, which puts every transaction into a one-hour window (07:00, 08:00, 09:00). `sum()` adds up the prices in each window. The result is a Series with two index levels: item and hour.

In [16]:
```
def calculate_hourly_revenue(df: pd.DataFrame) -> pd.DataFrame:
    """Calculate the total price for each item in one-hour intervals"""

    hourly_revenue = (
        df.groupby("item")
        .resample("1h")["price"]
        .sum()
        .rename("total_revenue")
    )

    return hourly_revenue
```

In [17]:
```
hourly = calculate_hourly_revenue(df)
print(hourly)
```

```
item        timestamp
Cappuccino  2026-10-15 07:00:00    41.42
            2026-10-15 08:00:00    25.15
Croissant   2026-10-15 07:00:00    18.96
            2026-10-15 08:00:00    34.49
            2026-10-15 09:00:00     4.16
Espresso    2026-10-15 07:00:00     3.86
            2026-10-15 08:00:00    21.30
            2026-10-15 09:00:00     2.79
Latte       2026-10-15 07:00:00    31.50
            2026-10-15 08:00:00    23.36
Muffin      2026-10-15 07:00:00    20.30
            2026-10-15 08:00:00    31.42
            2026-10-15 09:00:00    14.92
Sandwich    2026-10-15 07:00:00    53.41
            2026-10-15 08:00:00    15.37
Name: total_revenue, dtype: float64
```

**What the test shows:** The test file gives 15 rows, not 6 items × 3 hours = 18. For each item, `resample` only covers the hours from its first to its last sale, so an item with no sales after 09:00 (e.g. Cappuccino) gets no 09:00 row. The 09:00 values are low because the logs end shortly after 09:00. In the full pipeline every item has sales in all three hours, so the output file has 18 entries.

## Task 4.3 – Pivot table: revenue by item × status

**Our solution:** `pivot_table` puts the items as rows and the statuses as columns, and sums the price in each cell. `fill_value=0` gives 0 instead of `NaN` when an item has no transactions with a status, and `reindex` makes sure that all three status columns always exist, in the same order.

In [18]:
```python
def create_status_pivot_table(df: pd.DataFrame) -> pd.DataFrame:
    """Calculate total transaction prices by item and status"""

    pivot_table = pd.pivot_table(
        df,
        values="price",
        index="item",
        columns="status",
        aggfunc="sum",
        fill_value=0
    )

    # Make sure all three status columns are present
    pivot_table = pivot_table.reindex(
        columns=["SUCCESS", "REFUND", "CANCELLED"],
        fill_value=0
    )

    return pivot_table
```

In [19]:
```python
# Test Task 4.3: Create and display the status pivot table

pivot_table = create_status_pivot_table(df)

print("Status Pivot Table:")
display(pivot_table)
```

```
Status Pivot Table:
```

| status | SUCCESS | REFUND | CANCELLED |
|---|---|---|---|
| item | | | |
| **Cappuccino** | 50.02 | 0.00 | 16.55 |
| **Croissant** | 34.99 | 11.44 | 11.18 |
| **Espresso** | 12.54 | 9.78 | 5.63 |
| **Latte** | 36.20 | 4.12 | 14.54 |
| **Muffin** | 49.90 | 3.22 | 13.52 |
| **Sandwich** | 45.54 | 7.67 | 15.57 |

**What the test shows:** The table has 6 items × 3 statuses. Cappuccino has 0.00 under REFUND because it had no refunds in this log file, which shows that `fill_value=0` works. SUCCESS is the largest column for every item, as expected, since about two thirds of all transactions are successful.

## Task 4.4 – Refund ratio per item

Uses `compute_refund_ratio` from Task 3.2.

*Sales count:* we use the number of `SUCCESS` transactions as the sales count, because refunded and cancelled transactions are not completed sales.

**Our solution:** We count the transactions per item and status with `groupby(["item", "status"]).size().unstack(fill_value=0)`, which gives one column per status. The SUCCESS and REFUND columns are passed to `compute_refund_ratio` from Task 3.2 as NumPy arrays, so the ratio is computed for all items at once.

In [20]:
```python
# Task 4.4: Calculate refund ratios

def calculate_item_refund_ratios(df: pd.DataFrame) -> pd.DataFrame:
    """Calculate the refund ratio for each item"""

    # Count transactions for each item and status
    counts = df.groupby(["item", "status"]).size().unstack(fill_value=0)

    # Make sure the required status columns exist
    counts = counts.reindex(
        columns=["SUCCESS", "REFUND", "CANCELLED"],
        fill_value=0
    )

    # Calculate refund ratios using the function from Task 3.2
    ratios = compute_refund_ratio(
        counts["SUCCESS"].to_numpy(),
        counts["REFUND"].to_numpy()
    )

    # Create a result table
    result = pd.DataFrame({
        "item": counts.index,
        "sales_count": counts["SUCCESS"].to_numpy(),
        "refund_count": counts["REFUND"].to_numpy(),
        "refund_ratio": ratios
    })

    return result
```

In [21]:
```python
# Test Task 4.4: Calculate and display refund ratios

refund_ratios = calculate_item_refund_ratios(df)

print("Refund Ratios per Item:")
display(refund_ratios)
```

```
Refund Ratios per Item:
```

| | item | sales_count | refund_count | refund_ratio |
|---|---|---|---|---|
| **0** | Cappuccino | 12 | 0 | 0.000000 |
| **1** | Croissant | 9 | 3 | 0.333333 |
| **2** | Espresso | 4 | 3 | 0.750000 |
| **3** | Latte | 8 | 1 | 0.125000 |
| **4** | Muffin | 14 | 1 | 0.071429 |
| **5** | Sandwich | 6 | 1 | 0.166667 |

**What the test shows:** In this log file, Espresso has the highest ratio (3 refunds / 4 sales = 0.75), and Cappuccino has no refunds, so its ratio is 0. The full pipeline in Task 5 uses all 720 transactions.

## Task 5 – Full workflow

1. Find log files + active register IDs → 2. read & parse → 3. DataFrame → 4. outlier cleaning → 5. NOK × 10 → 6. reports → 7. save the 4 output files.

**Our solution:** `run_pipeline` calls the functions from Tasks 1–4 in the order given in the assignment, and keeps only transactions from active registers. Outliers are removed before the NOK prices are multiplied by 10, so the 3σ rule works on prices that are all on the same scale. Each report is then created and saved in `output/`. For the JSON file, the timestamps are converted to text first, so they are readable (by default pandas would write them as milliseconds).

In [22]:
```python
def run_pipeline() -> None:
    """Read, clean, analyse, and save all coffee shop transactions"""

    # Create the output folder if it does not exist
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    # Find all log files
    log_files = find_log_files(LOG_ROOT)
    print(f"Found {len(log_files)} log files.")

    # Read the store configuration and find active registers
    config = json.loads(CONFIG_FILE.read_text(encoding="utf-8"))
    active_registers = find_active_registers(config)

    print(f"Found {len(active_registers)} active registers.")

    # Read the log files and collect valid transactions
    records = []

    for file_path in log_files:
        text, encoding_used = read_text_file(file_path)
        file_records = parse_log(text)

        # Keep transactions from active registers only
        for record in file_records:
            if record["register_id"] in active_registers:
                records.append(record)

    print(f"Parsed {len(records)} active transactions.")

    # Create the initial DataFrame
    df = create_dataframe(records)

    if df.empty:
        raise ValueError("No valid transactions were found.")

    # Remove price outliers using the function from Task 3.1
    df["price"] = remove_price_outliers(df["price"].to_numpy())

    # Correct prices recorded in NOK
    df.loc[df["currency"] == "NOK", "price"] *= 10.0

    # Save the cleaned transactions
    df.reset_index().to_csv(
        OUTPUT_DIR / "cleaned_transactions.csv",
        index=False
    )

    # Calculate hourly revenue
    hourly_revenue = calculate_hourly_revenue(df)

    # Convert the result into a format suitable for JSON
    hourly_data = hourly_revenue.reset_index()
    hourly_data["timestamp"] = hourly_data["timestamp"].astype(str)

    hourly_data.to_json(
        OUTPUT_DIR / "hourly_revenue.json",
        orient="records",
        indent=2
    )

    # Create and save the status pivot table
    pivot_table = create_status_pivot_table(df)

    pivot_table.to_csv(
        OUTPUT_DIR / "status_pivot_table.csv"
    )

    # Calculate and save refund ratios
    refund_ratios = calculate_item_refund_ratios(df)

    refund_ratios.to_csv(
        OUTPUT_DIR / "item_refund_ratios.csv",
```

```
        index=False
    )

    print("All reports have been saved successfully.")
    print(f"Output folder: {OUTPUT_DIR}")
```

In [23]:
```
# Run the complete pipeline
run_pipeline()
```

```
Found 9 log files.
Found 3 active registers.
Parsed 720 active transactions.
All reports have been saved successfully.
Output folder: output
```

In [24]:
```
# Check the generated output files

print("Generated files:")

for file_path in OUTPUT_DIR.iterdir():
    if file_path.is_file():
        print(file_path.name)

# Read and inspect the cleaned transactions
cleaned_df = pd.read_csv(OUTPUT_DIR / "cleaned_transactions.csv")
print(cleaned_df.head())

# Read and inspect the pivot table
pivot_df = pd.read_csv(OUTPUT_DIR / "status_pivot_table.csv")
print(pivot_df.head())

# Read and inspect the refund ratios
refund_df = pd.read_csv(OUTPUT_DIR / "item_refund_ratios.csv")
print(refund_df.head())

# Read and inspect the hourly revenue report
with open(OUTPUT_DIR / "hourly_revenue.json", encoding="utf-8") as file:
    hourly_data = json.load(file)

print(hourly_data[:5])
```

```
Generated files:
hourly_revenue.json
item_refund_ratios.csv
status_pivot_table.csv
cleaned_transactions.csv
             timestamp register_id        item  price currency     status
0  2026-10-15 07:00:16      POS_02    Sandwich   7.16      USD    SUCCESS
1  2026-10-15 07:00:29      POS_02       Latte   4.61      USD     REFUND
2  2026-10-15 07:01:07      POS_02    Sandwich   7.32      USD    SUCCESS
3  2026-10-15 07:01:10      POS_02      Muffin  38.40      NOK    SUCCESS
4  2026-10-15 07:01:21      POS_01  Cappuccino   3.83      USD  CANCELLED
          item  SUCCESS  REFUND  CANCELLED
0  Cappuccino  1194.81  334.40    219.000
1   Croissant  1442.59  452.61    384.915
2    Espresso  1018.94  342.54    233.875
3       Latte  1496.83  237.77    273.855
4      Muffin  1311.65  181.32    266.840
          item  sales_count  refund_count  refund_ratio
0  Cappuccino           75            17      0.226667
1   Croissant           90            25      0.277778
2    Espresso           70            26      0.371429
3       Latte           66            17      0.257576
4      Muffin           94            15      0.159574
[{'item': 'Cappuccino', 'timestamp': '2026-10-15 07:00:00', 'total_revenue': 849.64}, {'item': 'Cappucc
ino', 'timestamp': '2026-10-15 08:00:00', 'total_revenue': 890.42}, {'item': 'Cappuccino', 'timestamp':
'2026-10-15 09:00:00', 'total_revenue': 8.15}, {'item': 'Croissant', 'timestamp': '2026-10-15 07:00:0
0', 'total_revenue': 906.005}, {'item': 'Croissant', 'timestamp': '2026-10-15 08:00:00', 'total_revenu
e': 1179.9}]
```

**What the output shows:** All 9 log files and 720 transactions are processed. No transactions are removed by the active-register filter, because POS_01–POS_03 are all active (see Task 2.2). After cleaning, prices range from 2.52 to 79.60. The mean NOK price is 45.15, about ten times the EUR (4.37) and USD (4.30) means, which shows that the currency correction worked. The hourly report has 18 entries (6 items × 3 hours). Sandwich has the highest SUCCESS revenue (2562.14), Espresso has the highest refund ratio (0.37) and Muffin the lowest (0.16).

Because NOK prices are about ten times larger than EUR and USD prices, NOK sales dominate the revenue totals. The totals mix currencies and should not be read as revenue in one currency.

## Discussion

The pipeline successfully processes the log files and generates four output reports. The encoding test shows that the file reader supports both UTF-8 and ISO-8859-1, although the generated log files contain only ASCII characters.

One limitation is that the hourly revenue report includes all transaction statuses and currencies. Refunds and cancelled transactions can therefore affect the totals, and different currencies are not directly comparable. The totals should not be interpreted as actual net revenue.

The 3σ rule helps identify extreme prices, but it may miss outliers or incorrectly flag valid prices. Outliers are removed before the NOK correction so that unusually high NOK values do not become even larger when multiplied by 10. However, this assumes that the outlier detection works correctly on the original prices.

Overall, the pipeline automates data cleaning and reporting, but the results must be interpreted with these limitations in mind.

## Summary of Gen AI use

Seline: AI Declaration

ChatGPT was used to provide step-by-step guidance throughout the assignment. It helped explain the tasks, clarify Python concepts and guide the development of functions using regular expressions, recursion, NumPy and Pandas.

AI assistance was also used to identify errors, test code, combine the functions into a complete workflow and write discussions of the results and limitations.

The purpose of AI use was to support understanding of the assignment and explain how the different parts of the code work together.

**Kachina:** I used Claude (Anthropic) mainly as a tutor. It gave me a step-by-step plan and a notebook template with hints, and explained the key concepts (encodings, regular expressions, recursion, outlier detection) with diagrams and examples. For Task 1, I asked for a full example solution, which I studied and used in the notebook. For Tasks 2 and 3, I wrote the functions myself by following the step-by-step hints in the template. The AI then checked my code and helped me find errors (missing imports and a syntax error). It also suggested docstrings and draft explanation texts, which I edited. I ran and tested all code myself.